

Engineering Notes: e-ring-voting v0.3

Onboarding for a new collaborator

Daniele Lin Niccolò Pagano

May 2026

These notes are for a colleague joining the project. The intended reader is a computer-science student who can read Python and knows the usual data-structures-and-algorithms toolkit, but has not taken a cryptography course. We build every concept from the ground up, including the parts that are not cryptography (the bulletin board, the state machine, the audit pipeline), and we are honest about which problems we have solved and which are still open.

By the end you should be able to read every file in the repository, explain what each one does and why, and decide which of the open problems you would attack first.

Contents

1	The thing we are building	3
2	How to read these notes	3
3	Cryptographic toolkit	4
3.1	Hash functions	4
3.2	Public-key cryptography in 60 seconds	4
3.3	Digital signatures	5
3.4	Groups (the algebra you actually need)	5
3.5	Hash-to-curve	6
3.6	Ring signatures	6
3.7	Trace tags: one-time traceable ring signatures	7
4	The system layer	8
4.1	The bulletin board (a JSON-Lines file)	8
4.2	The hash chain	9
4.3	Record types	9
4.4	The state machine	10
5	Decentralisation	10
5.1	The publisher cohort: t -of- N	10
5.2	The witness federation: k -of- M	11
6	The principals	12
7	The audit algorithm	12
8	Worked example	13

9	Architecture B: the proof-of-work chain	15
9.1	The block format	15
9.2	The PoW puzzle	16
9.3	The eVote ledger	16
9.4	The eight transaction kinds	17
9.5	Validation rules	17
9.6	The chain audit	18
9.7	A chain-side worked example	18
9.8	When each architecture wins	19
10	Codebase map	20
11	Dev environment	20
12	Open problems — where you would contribute	20
13	Critical assessment: is it “finally” anonymous and verifiable?	21
13.1	The precise anonymity claim	22
13.2	The precise verifiability claim	22
13.3	The honest pitch	23
A	Glossary	23

1 The thing we are building

A small program that lets a group of people vote on something and gives two guarantees that look like they contradict each other:

- **Nobody knows how you voted.** Not the organisers, not the developers, not anyone who reads the public record of the election.
- **Anyone can verify the count.** Given the public record, you can re-compute the result yourself and refuse to accept a different one.

The contradiction is only apparent. The system makes *each individual ballot* look like a sealed envelope (you can prove a legitimate voter signed it, but not which one), while *the pile of all envelopes* can be tallied honestly by anyone. The trick that makes this work is a cryptographic primitive called a *one-time traceable ring signature* (OTRS), which we describe in section 3.6.

The project has two layers, and the second layer ships in *two interchangeable architectures*:

1. A clean implementation of OTRS (`otrs/`, ~ 1.1 k LOC).
2. A voting system that uses OTRS to run real elections, in two forms with identical cryptography and record schema:
 - **Architecture A** (`voting/`, ~ 1.8 k LOC): a federated hash-chained log signed by a t -of- N publisher cohort with a k -of- M witness federation. The Certificate-Transparency-style path.
 - **Architecture B** (`chain/`, ~ 1.4 k LOC): a Bitcoin-shaped proof-of-work chain with permissionless mining, an eVote ledger, and typed voting transactions. The cryptocurrency-style path.

These notes spend most of their pages on Architecture A because it is the simpler primitive and pedagogically cleaner. Architecture B is introduced in section 9.8; everything in the audit and crypto sections applies unchanged. Total (≈ 4.4 k LOC + tests) is still small enough to fit in your head.

Concrete scale. Tens to a few hundred voters (university councils, union boards, neighbourhood associations, small political parties). Not nation-scale; we will say where the limits are and what would need to change.

2 How to read these notes

Section 3 is the cryptographic background. Read it even if you think you know hashes and signatures, because we introduce *groups*, *ring signatures*, and *trace tags*, which you probably haven't seen.

Section 4 is the system layer: how the ledger works, what entries look like, how the state machine is enforced. This is the part of the project that is just careful software engineering — no exotic crypto required.

Section 5 explains why and how the system is decentralised (threshold signing + a witness federation).

Section 7 explains the audit algorithm step by step — this is the function that makes “verifiable” actually true.

Section 8 walks through a complete election with concrete data.

Section 10 maps every concept to the file that implements it.

Section 12 lists the open problems — this is where you would contribute. Section 13 is the honest critical assessment of whether we have “finally” built an anonymous and verifiable voting system.

If you only have one hour, read sections 1, 3 (bulletin board), 5 (decentralisation), 8 (worked example), and 11 (open problems).

3 Cryptographic toolkit

We build the cryptographic vocabulary one concept at a time. Each sub-section introduces one idea, says what it is good for, and shows the math last (because the math becomes much easier to read once you know what it is trying to do).

3.1 Hash functions

A *hash function* is a deterministic recipe that maps any input (a string, a file, a number) to a fixed-size output. We use SHA-256, which produces 32 bytes.

Intuition.

A fingerprint. Two different inputs almost certainly produce two different fingerprints. Given a fingerprint, you cannot recover the input.

For our purposes a hash function H has three properties:

- **Deterministic.** $H(x) = H(x)$ for the same x .
- **Collision-resistant.** It is computationally infeasible to find $x \neq y$ with $H(x) = H(y)$.
- **One-way.** Given $H(x)$ you cannot find x (other than by guessing).

We use hashes for two jobs:

- To chain the ledger entries (each entry contains $h_{prev} = H(\text{previous entry})$, so tampering with history is detectable).
- To derive challenges inside the OTRS scheme. When the OTRS construction needs a “random” value that depends on a transcript (so all parties agree on it), we use a hash of the transcript. This trick has a name — the *Fiat-Shamir transform* — but you don’t need to remember the name to use it.

3.2 Public-key cryptography in 60 seconds

In symmetric cryptography Alice and Bob share a secret key and use it to encrypt and decrypt. In *public-key* cryptography each person has two keys:

- a *secret key* sk (kept private);
- a *public key* pk (published).

The two are mathematically linked. Given sk , computing pk is easy. Given pk , computing sk is computationally infeasible.

Public-key cryptography lets you do two kinds of operation:

1. *Encrypt to Alice* using her pk ; only her sk decrypts. (We don’t use this in this project.)
2. *Sign with Alice’s sk* ; anyone with her pk can verify the signature. This is what we use everywhere.

3.3 Digital signatures

A digital signature is a chunk of bytes that proves a message was endorsed by the holder of a specific secret key.

Intuition.

A wax seal on a letter. The seal proves the king sent it. If you change the letter, the seal breaks. Only the king has the stamp.

Formally, a signature scheme is three algorithms:

```
KeyGen() → (sk, pk)
Sign(sk, m) → σ
Verify(pk, m, σ) → {true, false}
```

We use **Ed25519** (a specific signature scheme) for every “ordinary” signature in the system: the organisers signing each entry on the ledger, the witnesses signing checkpoints, the attestations.

Ed25519 signatures are 64 bytes. Public keys are 32 bytes. They are fast (microseconds per verification) and have audited implementations. We use the `cryptography` Python library (which wraps OpenSSL).

Why two schemes?.

Voters do *not* sign their ballots with Ed25519. They use a different scheme called a *ring signature* that hides which voter signed (section 3.6). Ed25519 is for the parts of the system where identity *is* supposed to be visible (the organisers, the witnesses).

3.4 Groups (the algebra you actually need)

Many cryptographic primitives use *groups*. This is one piece of abstract algebra that you can’t completely avoid, but the working vocabulary is small.

Intuition.

A group is a set $\mathbb{G}$ of objects that you can “add” together, where adding behaves like ordinary addition: there is an identity element 0, every element has an inverse, and $(a+b)+c = a+(b+c)$.

If you can repeatedly add an element to itself, you can also “multiply” it by an integer: $g + g + g = 3 \cdot g$. So a group has scalar multiplication by integers.

For us the relevant group is **Ristretto255**. The set has $\approx 2^{252}$ elements. The technical details don’t matter; what matters is the operations and their cost:

Operation	Notation	Speed
group element + group element	$P + Q$	fast (microseconds)
integer $\times$ group element	$a \cdot P$	a few hundred microseconds
hash bytes $\rightarrow$ group element	$H(x) \in \mathbb{G}$	microseconds
discrete logarithm	given $P, a \cdot P$, find a	infeasible ($> 2^{128}$ ops)

The last row is the security assumption all our cryptography stands on. We pick a generator $g \in \mathbb{G}$ (a fixed, public element), and we treat $g, 2g, 3g, \dots$ as a one-way “ladder”: you can climb up easily, but given a rung you cannot find the step number.

This lets us define a *public key* as $pk = x \cdot g$, where x is a random integer. The secret key is x . Given pk , recovering x would require solving the discrete logarithm, which we believe is hard.

Notation warning.

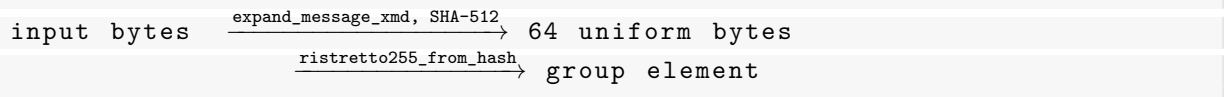
Cryptography papers traditionally use multiplicative notation (g^x instead of $x \cdot g$). We follow that convention in the rest of these notes because the OTRS paper does. So h^{x_i} means “add h to itself x_i times” in the additive view, or “raise h to the x_i -th power” in the multiplicative view. Either way, it’s a group element.

A *group order* q is the number of distinct elements in $\mathbb{G}$. For Ristretto255, $q = 2^{252} + 27742317777372353535851937790883648493$. Adding q copies of any element gives the identity. The exponents we work with are integers mod q , which we write $\mathbb{Z}_q$.

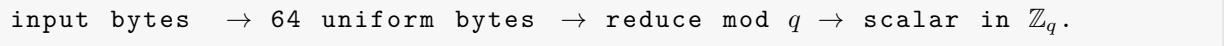
3.5 Hash-to-curve

The OTRS scheme needs to convert arbitrary bytes into group elements in a deterministic, well-mixed way. (Concretely: “the issue identifier mixed with the ring of voter pks” must become a group element h that everyone in the system agrees on.) The technique is called *hash-to-curve*.

We use the random-oracle suite from RFC 9380 [2]:



For hashing to scalars (integers mod q) we use the same expander followed by a reduction:



Both functions are pinned to specific *domain-separation tags* that begin with **OTRS-v1-RISTRETTO255-XMD:SHA-512-...** so that changing the algebra forces a protocol version bump.

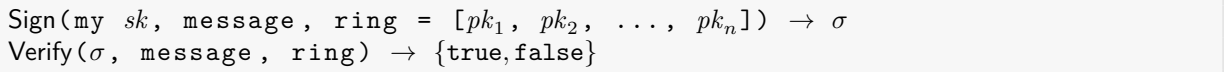
3.6 Ring signatures

This is where things get interesting. A *ring signature* lets you sign on behalf of a set of public keys (the “ring”) without revealing which key was used.

Intuition.

A sealed envelope labelled “one of these n people signed this”. You can verify the seal. You cannot tell who broke it open.

Concretely the public interface looks like this:



And the security property is:

Anyone who verifies σ learns “one of the keys in the ring signed this” — and nothing more.

The classical construction (Rivest, Shamir, Tauman, 2001) works like this in spirit:

1. The real signer is at position i and knows sk_i .
2. For every other position $j \neq i$, the signer picks *random* values that look like a partial proof.
3. Those random values, together, leave one missing piece in a global equation.

4. The signer uses their secret sk_i to compute the missing piece so that the equation balances.

From outside, all n positions look identical — random-looking data tied together by an equation that only closes if at least one secret key was used. Anyone can verify “yes, someone in this ring signed” but the position is hidden.

The new problem this creates. If a ring signature is perfectly anonymous, what stops the same voter from signing 100 ballots? Each one verifies fine; each one is anonymous. This is the sub-problem the OTRS construction solves.

3.7 Trace tags: one-time traceable ring signatures

The breakthrough in Scafuro and Zhang’s 2021 paper [1] is to add a *deterministic* tag to every signature, tied to $(signer, issue)$, in a way that:

- Looking at a single tag, you cannot tell whose it is.
- Two signatures by the same voter on the *same issue* produce equal trace tags.
- Comparing two tags reveals nothing else.

Intuition.

Imagine an envelope that turns transparent only if you mail two of them. One envelope = anonymous. Two envelopes by the same person = publicly transparent (you learn who sent both).

The math. Fix an issue I (the election identifier) and a ring $R = (pk_1, \dots, pk_n)$. Compute $h = H_0(I \| R)$ — a group element that depends only on the election and the ring, not on the message. The *trace tag* for voter i (secret key x_i , public key $pk_i = g^{x_i}$) is

$$\sigma_i = h^{x_i}.$$

Notice: σ_i depends only on (h, x_i) which depend only on (I, R, x_i) . The message m does not appear. So if voter i signs two messages $m_1 \neq m_2$ on the same election, both signatures embed the same σ_i .

How does the signature embed σ_i without leaking the position i ? It encodes the trace tag through two “signature points” A_0 and A_1 that satisfy

$$\sigma_j = A_0 \cdot A_1^j \quad \text{for every } j \text{ in the ring.}$$

A_0 depends on the message (so it’s different across messages); A_1 is chosen by the signer to make the equation hold at $j = i$ (see the full construction in the OTRS paper). Anyone verifying the signature computes the full vector $(\sigma_1, \dots, \sigma_n)$ from A_0, A_1 . Each σ_j looks like a random element. *Only* when you have two signatures and compare component-by-component does the trace tag’s invariance show up: the real signer’s column matches.

The trace algorithm, then, is just: compare the two σ vectors position by position. Exactly one matching position $\Rightarrow$ that position is the double-signer. Zero positions matching $\Rightarrow$ two independent signers. All positions matching $\Rightarrow$ the two signatures are identical (a replay).

The full signing algorithm. For completeness, the signature $\sigma = (A_1, c_1, \dots, c_n, z_1, \dots, z_n)$ is produced as follows. The signer:

1. Sets $h \leftarrow H_0(I\|R)$ and $\sigma_i \leftarrow h^{x_i}$ (the trace tag).
2. Sets $A_0 \leftarrow H_1(I\|R\|m)$.
3. Computes $A_1 \leftarrow (\sigma_i/A_0)^{1/i}$ so that $A_0 \cdot A_1^j$ correctly gives σ_i at $j = i$ and can be used by the verifier at every other j .
4. For each $j \neq i$ picks random $c_j, z_j \xleftarrow{\$} \mathbb{Z}_q$ and computes “simulated” proof points $a_j \leftarrow g^{z_j} p k_j^{c_j}$ and $b_j \leftarrow h^{z_j} \sigma_j^{c_j}$.
5. For $j = i$ picks random $w \xleftarrow{\$} \mathbb{Z}_q$ and sets $a_i \leftarrow g^w, b_i \leftarrow h^w$.
6. Computes the global challenge $c^* \leftarrow H_2(I\|R\|A_0\|A_1\|a_{1..n}\|b_{1..n})$.
7. Sets the missing piece $c_i \leftarrow c^* - \sum_{j \neq i} c_j$ and $z_i \leftarrow w - c_i x_i$.

The verifier recomputes everything and checks $\sum_j c_j \equiv c^* \pmod{q}$. The signature size is $32 + 4 + 64n$ bytes (A_1 , length prefix, $2n$ scalars).

This is the cryptographic core. In code: `otrs/otrs.py`. Read the functions `sign`, `verify`, `trace`; the math above maps line-for-line.

4 The system layer

The crypto in section 3 gives us one-vote-per-voter with anonymity. Turning that into an election system requires a few more pieces. None of these are cryptographic — they are just careful software engineering.

4.1 The bulletin board (a JSON-Lines file)

The whole history of an election is stored in one file. Each line of the file is one JSON object called an *entry*. Entries are append-only: once written, they don’t get edited.

Intuition.

A notebook in a public square. Anyone can read it. Only the organisers can write. Every page is dated, signed, and links to the previous page.

An entry has the following shape:

```
{
  "index":      0,
  "prev_hash":  "AAAA...",
  "timestamp":  1716580800,
  "payload":    "eyJraW5kIjogImVsZWNOaW9uX3NldHVwIiwgLi4u",
  "publisher_sigs": [
    {"i": 0, "sig": "Cg7q..."},
    {"i": 1, "sig": "Tp1m..."}
  ]
}
```

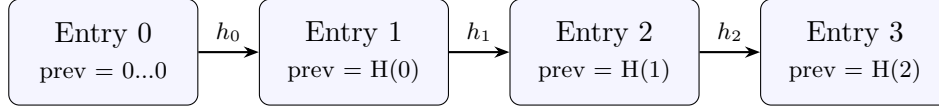
The fields:

- `index` — entry number, starting at 0 and contiguous.

- `prev_hash` — SHA-256 of the previous entry (the *hash chain*; explained below).
- `timestamp` — Unix seconds, monotonically non-decreasing.
- `payload` — base64-encoded JSON of one of six record types (section 4.3).
- `publisher_sigs` — a list of (i, σ) pairs, each a signature by cohort member i . We require at least t distinct members to have signed (the *threshold*, section 5.1).

4.2 The hash chain

The `prev_hash` field links each entry to the one before it.



Why this is useful: if anyone tampers with entry 1 (changes a byte), its hash h_1 changes. Entry 2’s `prev_hash` still claims the old value, so the chain is broken. The auditor catches this in $O(N)$ time during verification.

This is the “blockchain-flavoured” bit.

A hash chain is the data structure underneath Bitcoin’s “blockchain”. We use the data structure — not the rest of Bitcoin. There is no proof-of-work, no mining, no currency, no consensus protocol. The right reference is Certificate Transparency [4], not Bitcoin. See section 9.8 for why we deliberately skip PoW.

The entry hash $H(\text{entry})$ is computed over the *content fields only*, not over the signatures:

$$H(\text{entry}) = \text{SHA256}(\text{index} \parallel \text{prev_hash} \parallel \text{timestamp} \parallel |\text{payload}| \parallel \text{payload}).$$

This means a cohort can collect more signatures on an entry over time without changing the chain. Convenient for asynchronous signing (section 5.1).

4.3 Record types

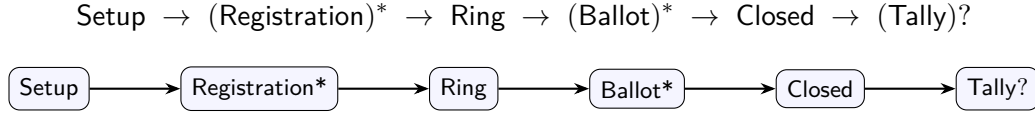
The `payload` of an entry is a JSON object with a `kind` field that identifies the record type. There are six:

Kind	Content
ElectionSetup	Genesis entry. Election ID (32 random bytes, also used as the OTRS issue I), title, options, schedule, cohort public keys, threshold t , optional witness keys and threshold k .
VoterRegistration	A voter’s OTRS public key plus a human-readable handle.
RingPublication	The ordered ring of all registered voter pks. After this, registration is closed and voting opens.
Ballot	A voter’s choice (one of the options) together with their OTRS signature.
VotingClosed	A marker. No ballots may appear after this.
TallyPublication	The cohort’s claimed final tally. Anyone can recompute and refute.

Code: `voting/records.py`. Each `kind` is a Python dataclass with `to_payload` / `from_obj` methods. The polymorphic parser `parse_payload` dispatches by `kind`.

4.4 The state machine

The legal sequence of records is



The auditor enforces this regex by walking the log linearly and keeping a state variable. Any out-of-order record is rejected. The state machine is the simplest possible enforcement of “ballots after ring, no ballots after close”.

5 Decentralisation

The system described so far has one weak spot: “the organiser”. What if the organiser:

- refuses to publish (the election stalls);
- goes offline (the election stalls);
- gets compromised (the attacker rewrites the result);
- lies about the tally (we catch them at audit, but it took us months to set up an election that’s now invalid).

Real elections solve this with *multiple organisers* and rules of the form “any decision requires the agreement of at least t of them”. We do the same digitally.

5.1 The publisher cohort: t -of- N

Instead of one organiser with one Ed25519 key, we have a *cohort* of N organisers, each holding their own key. Every log entry must be co-signed by at least t distinct cohort members to be valid.

```
ElectionSetup pins:
  cohort_pks = [pk0, pk1, ..., pkN-1]
  threshold  = t

Entry has:
  publisher_sigs = [(i1, σ1), (i2, σ2), ...]
                  # at least t distinct indices, each sig valid under
                  cohort_pks[i]
```

This gives us two guarantees:

- **Liveness against $N - t$ unavailable members.** Up to $N - t$ cohort members can be offline or refusing to cooperate; the remaining t can still proceed.
- **Soundness against $t - 1$ corrupted members.** A minority faction smaller than t cannot append on its own.

Common configurations:

Pick a configuration that matches your trust model: larger t trades liveness for soundness.

t -of- N	tolerates offline	tolerates corrupt
2-of-3	1	1
3-of-5	2	2
4-of-7	3	3
7-of-10	3	6

Asynchronous signing. Cohort members typically live on different machines. The *pending-entry* protocol lets them sign over time:

1. Any cohort member calls `propose_entry(log, payload)` to create a sidecar file `<log>.pending`.
2. Each cohort member, in any order, calls `cosign_pending` to add their signature to the sidecar.
3. Once the sidecar has $\geq t$ signatures, anyone calls `commit_pending` to move it into the main log.

Code: `voting/log.py`. CLI: `evote propose, evote sign-pending, evote commit-pending`.

5.2 The witness federation: k -of- M

The cohort layer protects against a corrupt minority. What if a corrupt *majority* ($\geq t$ malicious members) collude to publish two different versions of the log to different observers?

This is called *equivocation*. Cohort signatures alone cannot prevent it (the colluding majority has enough signing power). The classical fix is to add an *independent* set of read-only checkers: *witnesses*.

A witness has their own Ed25519 keypair. Their job is to:

1. Fetch the bulletin board.
2. Verify it from scratch under the cohort identity.
3. Emit a signed *checkpoint* attesting to what they saw: “at index i , the head hash is h ”.

```
Checkpoint = (log_index, head_hash, witness_index, sig)
sig = Ed25519.Sign(witness_sk, "otrs-witness-checkpoint-v1" ||
    log_index || head_hash || witness_index)
```

Checkpoints accumulate in a sidecar file `<log>.witnesses`.

What property does this buy us? If a corrupted majority shows two different logs to two different honest witnesses, the two checkpoints they emit will have the *same log_index* but *different head_hash*. Anyone who collects both checkpoints has public, undeniable proof that the cohort split. The auditor surfaces this as **EQUIVOCATION**, and the election is invalidated.

What the auditor requires. If the setup declared witnesses, the auditor:

- verifies every checkpoint’s signature;
- checks that no two checkpoints sign the same index with different hashes (no equivocation);
- requires at least k distinct witnesses to have co-signed *some* log index (the *witness threshold*).

Code: `voting/witness.py`. CLI: `evote witness-keygen, evote witness-checkpoint`.

6 The principals

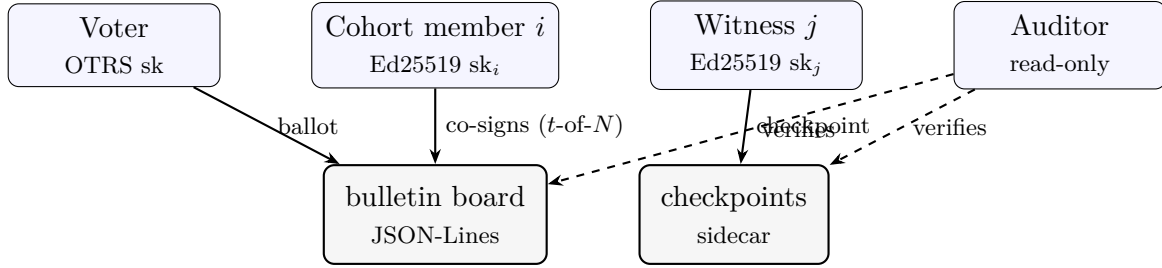
Four kinds of party in the system:

Cohort member. Holds one Ed25519 secret key. Can propose, co-sign, and commit entries to the main log. Trusted (with the rest of the cohort) to not collude past the threshold. Code: `voting/manager.py`.

Voter. Holds one Ristretto255 OTRS secret key. Can read the log, build a ballot, and submit it. Identity is verified out-of-band by the cohort during registration; after that the voter’s actions are anonymous within the ring. Code: `voting/voter.py`.

Witness. Holds one Ed25519 secret key. Cannot write to the main log. Periodically fetches, verifies, and emits checkpoints to the witness sidecar. Code: `voting/witness.py`.

Auditor. Anyone. Read-only. Holds no keys. Bootstraps the cohort and witness identities from the Setup entry, then runs the audit pipeline. Code: `voting/auditor.py`.



Voters do not need to know who the cohort members are by name; they need only the cohort *public keys*, which they read from the genesis Setup entry. Auditors do not need any private state; they only need the log files.

7 The audit algorithm

The single most important function in the project is `voting.auditor.audit`. It is what makes “verifiable” actually true. It runs in seven steps. If any check fails it raises `AuditError` and the election is invalid.

Step 1: bootstrap. Parse entry 0 as an `ElectionSetup`. Extract: the election ID, the cohort pks, the threshold t , the witness pks, the witness threshold k . All subsequent checks use this Setup as ground truth.

Step 2: chain integrity. For every entry $i = 0, 1, 2, \dots$:

- the index field equals i ;
- `prev_hash` equals the SHA-256 of entry $i - 1$ (or all zeros for $i = 0$);
- there are $\geq t$ distinct signer indices in `publisher_sigs`, and every signature verifies under the cohort key at the claimed index;
- the timestamp is $\geq$ the previous entry’s timestamp.

Step 3: state-machine. Walk the entries enforcing the regex of section 4.4. Any record out of state is rejected.

Step 4: per-record validity.

- Every voter pk in a `VoterRegistration` record must be a valid Ristretto255 group element.
- The `RingPublication`'s ring must be exactly the multiset of voter pks in the prior `VoterRegistration` records (no extras, no missing).
- Every `Ballot`'s choice must appear in the `ElectionSetup` options.
- Every ballot's OTRS signature must verify against the election ID and the ring.
- Ballot timestamps must lie within `[voting_open, voting_close]`.

Step 5: witness federation (if declared).

- Every checkpoint's signature verifies under the claimed witness key.
- No two checkpoints sign the same index with different head hashes (no `EQUIVOCATION`).
- At least k distinct witnesses must agree on at least one index.

Step 6: tally. For each ballot k compute the OTRS column vector $(\sigma_1^{(k)}, \dots, \sigma_n^{(k)})$ from its signature points $A_0^{(k)}, A_1^{(k)}$. Then:

- Two ballots share a signer iff their column vectors collide at some position j .
- Group all ballots that share collision positions. Each group corresponds to one voter.
- If a group has one distinct choice, count it once.
- If a group has two or more distinct choices, that voter *double-signed*: reject all of their ballots and emit their pk in the culprit list.

The probability that two ballots by *different* voters spuriously share a column is bounded by $\binom{B}{2} \cdot 2^{-252}$ for B ballots — cryptographically negligible.

Step 7: tally claim verification. If the log contains a `TallyPublication`, compare the auditor-computed tally to the cohort's claim. A mismatch fails the audit. (The CLI exits non-zero, which a script can check.)

Output. An `AuditReport` object containing the canonical tally, the per-ballot accepted/rejected status, the list of double-sign culprit pks, and the witness-co-signing status. The CLI prints all of this.

8 Worked example

A four-voter election run by three faculty cohort members and two witnesses. Concrete file by file.

Setup. Three Ed25519 keypairs for the cohort (Profs. Adami, Bianchi, Conti), threshold 2-of-3. Two for the witnesses (Dean, Ombudsman), threshold 1-of-2 (loose for a small demo). Four voters (Alice, Bob, Carol, Dave), each with their own Ristretto255 OTRS keypair.

Entry 0 (genesis).

```
{
  "index": 0, "prev_hash": "AAAA...", "timestamp": 1716580800,
  "payload": "<base64 of: {kind: election_setup, election_id: 'a4b2...',
    title: 'Cafeteria midnight closing', options: ['yes','no'],
    cohort_pks_b64: [Adami_pk, Bianchi_pk, Conti_pk], threshold: 2,
    witness_pks_b64: [Dean_pk, Ombuds_pk], witness_threshold: 1,
    registration_close: ..., voting_open: ..., voting_close: ...}>",
  "publisher_sigs": [
    {"i": 0, "sig": "<Adami's sig>"},
    {"i": 1, "sig": "<Bianchi's sig>"}
  ]
}
```

Entries 1–4 (registrations). Each is a VoterRegistration co-signed by two cohort members (possibly a different pair per voter — the subset can rotate).

Entry 5 (ring publication). The four voter pks, sorted by raw bytes, co-signed by two cohort members.

Entries 6–9 (ballots). Each voter’s OTRS-signed ballot. The cohort just adds it to the log — the cohort does not verify the OTRS signature here, because the auditor will. (If they did, they’d reveal that they trusted the voter; this way they don’t even need to look at the ballot to publish it.)

Entry 10 (voting closed). VotingClosed marker, co-signed by two cohort members.

Witness checkpoints. Dean and Ombudsman each independently fetch the log, verify the chain, and emit a checkpoint at (log_index=10, head_hash=H) signed under their own key. Both agree on the same *H* — the cohort was honest about what the head looks like.

Audit output.

```
$ python3 -m voting.cli audit --log log.jsonl

election:                Cafeteria midnight closing
election_id:             a4b2...
cohort:                  2-of-3
witnesses:               1-of-2; valid checkpoints: 2; latest co-signed
    ↪ idx: 10
ring size:               4
accepted ballots:        4
rejected ballots:         0
tally:
  no                      1
  yes                     3
double-sign culprits:    0
```

What if Alice tried to vote twice? She signs two ballots with different choices. Both go on the log. At audit time, OTRS Trace fires: Alice’s column matches in both ballots. Both ballots are rejected; Alice’s pk is in the culprit list. The link from pk to “Alice the human” lives in her VoterRegistration entry (handle “alice”), so the cohort can follow up; but the cryptography itself is at the pk level.

What if the cohort lies about the tally? They publish a `TallyPublication` with the wrong numbers. The auditor recomputes from the actual ballots, finds the mismatch, exits non-zero. The lie is publicly visible.

What if a corrupted majority equivocates? They show ledger A to the Dean and ledger B (with one ballot different) to the Ombudsman. Both are validly cohort-signed. The Dean signs `Checkpoint(idx=10, hash=A)`, the Ombudsman signs `Checkpoint(idx=10, hash=B)`. Anyone who collects both checkpoints sees the same index with two different hashes: undeniable proof. The auditor raises `AuditError("EQUIVOCATION")`.

9 Architecture B: the proof-of-work chain

So far every section has described Architecture A — the federated bulletin board in `voting/`. The repository ships a second architecture in `chain/` that uses the *same* cryptography (OTRS, exactly), the *same* record types (`voting/records.py`, unchanged), and the *same* audit algorithm for the tally step — but replaces the cohort-signed log with a Bitcoin-shaped proof-of-work chain. The point of building both is to make the trade-off concrete, not theoretical: with the same primitive underneath, what does the storage layer actually buy you?

This section walks through the chain at the same intuition-then-math pace as the rest of the notes. If you understood A, B will feel like a swap of one building block — the storage layer — with the rest of the structure preserved.

Intuition.

A chain is the same idea as the bulletin board (an append-only, hash-chained record), with two differences that come from the cryptocurrency lineage: (i) anyone can append, but appending is costly (you have to solve a puzzle); (ii) there is an internal token economy — eVotes — so that the people doing the appending (miners) get paid.

9.1 The block format

A block is a fixed 121-byte header plus a variable-length list of transactions. The header is what gets hashed for the PoW puzzle:

Field	Bytes
<code>prev_hash</code> (SHA-256 of parent's header)	32
<code>height</code> (big-endian)	8
<code>difficulty</code> (1 byte, 0–255)	1
<code>timestamp</code> (Unix seconds, big-endian)	8
<code>miner_pk</code> (Ed25519 raw)	32
<code>tx_root</code> (SHA-256 of canonical tx bytes)	32
<code>nonce</code> (big-endian)	8

The transactions themselves live alongside the header (not inside it). They are committed to via `tx_root`, which is the SHA-256 hash of the concatenation of each transaction's canonical bytes. So if anyone tampers with a transaction body, `tx_root` no longer matches and the block fails validation.

Heads up.

This is a Merkle-flat root, not a Merkle tree. Merkle trees buy you *inclusion proofs* (proving a specific tx is in a block without sending the whole block). Our blocks are small enough that we ship the whole tx list at audit time, so the simpler flat hash is sufficient. Adding a Merkle tree is one of the obvious follow-ups if you want light clients.

9.2 The PoW puzzle

The proof-of-work requirement is the simplest possible:

$$\text{leading_zero_bits}(\text{SHA-256}(\text{header})) \geq \text{difficulty}.$$

The **nonce** field of the header is the only thing the miner mutates; they try $\text{nonce} = 0, 1, 2, \dots$ until the hash has enough leading zeros.

Intuition.

SHA-256 is unpredictable: changing the nonce by 1 produces a hash that looks completely different. With probability 2^{-d} a random nonce produces a hash with d leading zero bits. So mining at difficulty d takes on average 2^d tries.

For the test suite we pin difficulty at 4–8 bits (16–256 expected hashes per block, sub-millisecond mining time on a laptop). For a production deployment Bitcoin-style difficulty would be ≈ 80 bits (10^{24} hashes per block, infeasible on a single CPU). The *difficulty adjustment* rule (in `chain/mining.py`) bumps difficulty up or down by 1 if the median inter-block interval over the last 11 blocks falls outside $[0.5T, 1.5T]$ around a target $T = 60$ seconds. This is exactly the legacy node’s logic from before this rewrite.

Cost vs. difficulty (measured). On the bench machine the single-process Python miner sustains $\approx 270,000$ SHA-256 hashes per second:

d	expected hashes	mine time / block
4	16	0.09 ms
8	256	0.95 ms
12	4096	12.6 ms
16	65536	264.6 ms

A native (Rust/C) miner on the same hardware would hit $\sim 50\text{M}$ hashes per second — $\sim 200,000\times$ faster — but the cost *ratio* between honest miners and any attacker is the same, and that ratio is what the security analysis turns on.

9.3 The eVote ledger

Every Ed25519 public key has an *account*:

$$\text{Account} = (\text{balance}, \text{nonce}).$$

balance is the eVote balance, an unsigned integer. **nonce** is the number of transactions this account has sent so far. The next transaction sent by this account *must* have **nonce** equal to the account’s current nonce; after applying the tx the account’s nonce increments by 1. This is the standard account-model replay defence (Ethereum uses the same).

Heads up.

Why not UTXO (Bitcoin’s model)? UTXO is better for parallelism and fungibility analysis; account is far simpler in Python and matches our use case (mostly account-to-miner fee flows, no anonymity plumbing). For an OTRS-voting prototype the win from UTXO is negligible.

The genesis block contains zero or more `coinbase` transactions that mint initial allocations. After that:

- Each non-genesis block contains *exactly one* `coinbase` as its first transaction;
- The coinbase amount must equal $block_reward + \sum fees$ across all other transactions in the block;
- Any other transaction failing validation rejects the entire block (state changes are atomic — the block is applied on a snapshot, committed only on success).

9.4 The eight transaction kinds

Two are *plumbing* (move eVotes around), six *wrap the voting record types* you already know from section 4.3.

Kind	Signed by	Wraps
<code>coinbase</code>	— (per-block, first tx)	mints reward + fees
<code>transfer</code>	sender	— (eVote payment)
<code>setup_poll</code>	poll creator	<code>ElectionSetup</code>
<code>register_voter</code>	sponsor	<code>VoterRegistration</code>
<code>publish_ring</code>	poll creator	<code>RingPublication</code>
<code>ballot</code>	<i>none</i>	<code>Ballot</code>
<code>close_poll</code>	poll creator	<code>VotingClosed</code>
<code>tally</code>	poll creator	<code>TallyPublication</code>

Issue.

Ballots are the only unsigned kind. They have no chain-layer sender, no Ed25519 signature, and pay no fee. This is *essential* for anonymity: a fee-bearing or sender-bearing ballot would expose the voter’s identity on chain. The validity check is purely the OTRS signature inside the ballot payload, which anyone can verify against the published ring.

The cost of this design is that miners receive no per-ballot incentive — they include ballots because the protocol rules say they must. In a hostile network you would compensate miners via the *poll setup fee* (escrowed by the poll creator), but the current prototype keeps the rule simple.

9.5 Validation rules

For each kind, `chain/state.py` enforces:

- **transfer**: sender’s nonce matches account, signature verifies under sender’s pk, balance covers sum-of-recipients plus fee.
- **setup_poll**: creator’s nonce + signature + fee OK; the embedded `ElectionSetup` parses and validates; `election_id` not already in use; creator becomes the named manager of the poll.

- **register_voter**: sponsor’s nonce + sig + fee OK; **election_id** exists and is in registration phase; block timestamp before **registration_close**; voter pk is a valid Ristretto point and not already registered.
- **publish_ring**: creator’s nonce + sig + fee OK; poll exists; *caller is the poll creator*; ring set equals the set of registered voter pks.
- **ballot**: poll exists and is in voting phase; block timestamp in [**voting_open**, **voting_close**]; choice in option set; OTRS signature verifies against the published ring.
- **close_poll, tally**: creator-only; phase guards; **close_poll** requires **block_timestamp** $\geq$ **voting_close**.

The poll-creator check is the chain’s analogue of A’s cohort: each poll has a single accountable identity that can publish phase transitions for it. The chain itself, however, is permissionless — anyone can mine the blocks.

9.6 The chain audit

The audit pipeline for B is almost identical to A’s (section 7), with one step swapped:

1. **Chain integrity.** Re-load the chain from disk. For each block: verify PoW, verify **prev_hash** chains back, verify timestamps strictly increase, verify **tx_root** matches transactions, run **apply_block** (which re-validates every signature, every nonce, every balance, every state-machine transition, and OTRS-verifies every ballot). *This is the step that differs from A; the rest is identical.*
2. **Per-poll tally.** For each poll in the resulting state, run the same σ -column algorithm as in section 7: bucket every ballot’s column tags, identify any signer who appears in two ballots with different choices (double-sign), aggregate the rest into the tally.
3. **Compare against any on-chain TallyPublication:** if present and the numbers disagree, the poll creator is lying. Audit exits non-zero.

By the time step (1) completes, the chain has been re-validated end to end; if you trust the result of step (1), the only remaining question is the tally, and that is deterministic from the ballots.

9.7 A chain-side worked example

Same cast of characters as section 8: Alice, Bob, Carol, with the addition of Mallory the miner. Three blocks per phase, one transaction per block (the CLI’s convention):

1. **Genesis.** Mallory’s node starts a chain. The genesis block contains a coinbase that mints 1000 eVotes to Alice (the project sponsor). Height 0. No PoW needed for genesis — there is no parent to anchor to.
2. **Setup.** Alice broadcasts a **setup_poll** transaction containing the **ElectionSetup** payload (title “Best Lunch”, options “pasta / pizza / focaccia”, timestamps). Fee: 5 eVotes. Mallory mines a block at height 1 containing this tx plus a coinbase for $50 + 5 = 55$ eVotes to herself. The PoW puzzle is satisfied; Mallory broadcasts the block.
3. **Three registrations.** Alice broadcasts three **register_voter** transactions (one per voter pk, fee 1 each). Mallory mines three blocks at heights 2, 3, 4. Alice’s balance is now $1000 - 5 - 3 = 992$.

4. **Ring publication.** Alice’s `publish_ring` tx freezes the ordered ring; mined into block 5. From this point the poll is in the `voting` phase; ballots become valid.
5. **Three ballots.** *Each voter* now broadcasts a `ballot` transaction — unsigned at the chain layer, OTRS-signed inside the payload. Mallory mines blocks 6, 7, 8 containing one ballot each. The ballots travel over the public network; their *contents* are public; but anonymity inside the ring of three still holds because the OTRS signature does not identify the signer, and there is no Ed25519 wrapper sender.
6. **Close.** Alice broadcasts `close_poll` once the voting window passes; mined into block 9.
7. **Audit.** Anyone with the chain file can run `python3 -m chain.cli audit -chain c.bin`. The replay confirms PoW + state-machine + OTRS-per-ballot; the σ -column tally counts 2 pasta vs. 1 pizza; no `TallyPublication` present, so the audit just prints the result.

Double-sign on chain. If Bob tries to vote twice (perhaps Mallory accepts the second tx into a later block), Bob’s two ballots both verify individually, but the auditor’s σ -column algorithm sees the same column tag appearing in two ballots with different `choice` values. Both Bob ballots are discarded; Bob’s OTRS public key is surfaced in the audit report. The trace algorithm is identical to A’s — it depends only on the ballots, not on how they were ordered.

Equivocation on chain. The chain has no analogue of A’s *witness federation*. The cheapest defence is the longest-chain rule itself: a $\geq 51\%$ miner can rewrite recent blocks (a *reorg*). Long-range reorgs of an abandoned chain are not even cheap — they are free. This is the structural weakness B inherits from PoW.

9.8 When each architecture wins

A wins when the election has a known accountable authority set — a university senate, a corporate board, an inter-institution treaty body. Finality is in seconds; there is no token economy; operating cost is bounded to running $N + M$ named parties. The cohort + witness model is the same one Certificate Transparency, Sigsum [5], and sigstore use for similar problems.

B wins when permissionless validator membership is a non-negotiable design constraint — no party, including the poll creator, is trusted to host the canonical record. The chain has no genesis-pinned trust set beyond the protocol rules; anyone with hashpower can produce blocks.

Neither solves coercion resistance (a voter who reveals x_i enables trace-tag matching in both systems), compromised voter devices (a malicious client signs whatever it wants), or post-quantum security. These live upstream of the storage layer.

Where B specifically loses. Finality is probabilistic (reorgs are always possible). There is no equivocation witness — if a $\geq 51\%$ miner rewrites history, no out-of-band cross-check flags it. Most importantly: the security budget per unit time is bounded by aggregate fees, while the attacker’s payoff scales with the largest poll on the chain at any moment. For elections of serious political stakes, rented-hashpower attacks become cost-effective even on a continuously-mined multi-poll chain. There is a serious academic paper [10] on this; the `chain`/-specific addendum in `paper/threat_model.md` §7 quantifies the bound.

End-to-end cost (measured). At $N = 50$ voters, Architecture A finishes a full election (N registrations, ring publication, N ballots, close, audit) in ≈ 1.1 s and produces a 324 KB bulletin board. Architecture B finishes the same election in ≈ 3.0 s and produces a 344 KB chain. The $\sim 1.6\times$ overhead and $\sim 6\%$ storage overhead come from PoW plus full transaction validation per block. Full tables are in the evaluation section of `paper/artifact.tex`.

10 Codebase map

File	What's in it / when to read it
<code>otrs/group.py</code>	Scalar, Point dataclasses; arithmetic over Ristretto255. Read when touching the math.
<code>otrs/hash.py</code> <code>otrs/otrs.py</code>	RFC 9380 <code>expand_message_xmd</code> , <code>hash_to_group</code> , <code>hash_to_scalar</code> . <code>keygen</code> , <code>sign</code> , <code>verify</code> , <code>trace</code> . The math in section 3.7 maps line-for-line.
<code>otrs/serialize.py</code> <code>otrs/_libsodium.py</code>	Canonical byte encodings. Touch if you ever change a wire format. cffi binding to libsodium ristretto255. The boundary to native code.
<code>voting/log.py</code> <code>voting/records.py</code>	<code>Entry</code> , <code>BulletinBoard</code> , <code>PublisherCohort</code> , pending-entry helpers. The six record types + canonical JSON.
<code>voting/manager.py</code> <code>voting/voter.py</code>	Cohort APIs: <code>setup</code> , <code>register</code> , <code>ring</code> , <code>ballot</code> , <code>close</code> , <code>tally</code> . Voter APIs: <code>key import/export</code> , <code>ballot construction</code> .
<code>voting/witness.py</code> <code>voting/auditor.py</code>	<code>Checkpoint</code> , <code>emit_checkpoint</code> , <code>verify_checkpoints</code> . <code>audit</code> — the seven-step pipeline of section 7.
<code>voting/cli.py</code>	The <code>evote</code> command-line tool.
<code>tests/test_otrs.py</code> <code>tests/test_log.py</code>	38 cryptographic tests (correctness + adversarial). 14 bulletin-board integrity tests.
<code>tests/test_election.py</code> <code>tests/test_threshold.py</code>	10 end-to-end election integration tests. 12 tests for threshold cohort and witnesses (incl. equivocation).

Suggested first-pass reading order: `otrs/otrs.py`, `voting/log.py`, `voting/records.py`, `voting/auditor.py`. Then the tests.

11 Dev environment

```

sudo apt install -y libsodium23 python3-cffi python3-pytest \
    python3-hypothesis python3-cryptography texlive-latex-recommended

# from repo root
python3 -m pytest -v                                # all 111 tests
    ↪ in <1s
python3 -m bench.bench_otrs --sizes 2,4,8,16,32,64  # OTRS microbench
make -C paper                                       # rebuild the
    ↪ artifact paper PDF
make -C docs                                       # rebuild THESE
    ↪ notes

```

A copy-paste end-to-end CLI demo is in the project `README.md`; running it takes about 30 seconds.

12 Open problems — where you would contribute

In roughly increasing order of difficulty:

1. **Tamarin / ProVerif model of the protocol layer.** Encode the state machine, the threshold cohort, and the witness federation as a process calculus and verify ballot anonymity + double-vote detection automatically. The state space is small. Estimated effort: 6–10 weeks. Output: a publishable artifact-track paper. **Highest value-to-effort ratio.**
2. **FROST threshold Ed25519.** Replace the per-entry vector of t separate Ed25519 signatures with a single 64-byte FROST signature [3]. Drops storage by $64(t - 1)$ bytes per entry. Engineering with real cryptography. Estimated effort: 8–12 weeks.
3. **Gossip protocol for log + checkpoint replication.** The cohort and witnesses should each hold a copy of the log and the checkpoint sidecar, with a gossip protocol that anchors heads across peers. An escrowed secondary cohort that activates on timeout addresses the “unanimously-malicious cohort” liveness gap. Estimated effort: 12–16 weeks.
4. **Browser voter client.** The current CLI is fine for developers and ridiculous for real voters. A WebAssembly build of the OTRS signer plus a small webapp that submits to the cohort over HTTPS makes the voter side realistic. Estimated effort: 8–12 weeks.
5. **Multi-election bulletin board.** Currently one file equals one election. A registry of past elections, replicated across the cohort, makes long-running organisations (universities, unions) practical. Estimated effort: 4–6 weeks.
6. **Logarithmic-size traceable rings.** A signature is currently $O(n)$ bytes. A Groth–Kohlweiss one-out-of-many proof [6] would give $O(\log n)$ size for the membership step. Combining it with the trace tag $\sigma_i = h^{x_i}$ in a way that still allows Trace to identify the position is, to our knowledge, *open* in the literature. Estimated effort: 6–12 months and probably co-author work with a crypto researcher.
7. **Receipt-freeness.** A voter who reveals x_i to a coercer can prove their vote by recomputing $\sigma_i = h^{x_i}$ and matching it on-chain. The fix is a designated-verifier re-encryption layer à la JCJ [7] or Civitas [8]. Stacking it on OTRS is non-trivial because the trace tag is the very thing that breaks receipt-freeness. Estimated effort: 12–18 months.
8. **EasyCrypt proofs of OTRS.** Re-prove the unforgeability / anonymity / traceability theorems of [1] in EasyCrypt and link the proofs to the implementation via a code-extraction tool. Machine-checked security for the cryptographic core. Estimated effort: 6–9 months with an EasyCrypt mentor.
9. **Post-quantum traceable ring.** The discrete-log security assumption falls to Shor’s algorithm. Lattice-based linkable rings exist (Raptor [9]); extending to traceability with one-time tags is the hard part. Estimated effort: 12+ months, research project.

Pick by goal.

- Master’s thesis aimed at a strong PhD program: item 1 or item 8.
- Industry security engineering: item 2, 3, or 4.
- Cryptography research career: item 6 or item 9, but get a co-advisor first.

13 Critical assessment: is it “finally” anonymous and verifiable?

Short answer: **yes, in a precise technical sense.** Long answer: you have to be able to recite the caveats. Both are in this section.

13.1 The precise anonymity claim

What we provide. Given the bulletin board, the published ring $R = (pk_1, \dots, pk_n)$, and one ballot’s OTRS signature σ on message m under issue I : no probabilistic polynomial-time adversary can determine which $pk_i \in R$ produced σ with probability non-negligibly better than $1/n$, assuming the discrete-log problem is hard in Ristretto255 and the random-oracle model holds for the hash functions.

Where this breaks down.

Issue 1 (Anonymity-set ceiling). The anonymity set is the ring R . For $n = 4$ a single ballot is hidden among four candidates — if a coercer knows the other three votes by other means, the fourth is exposed by elimination. Mitigation: use larger rings.

Issue 2 (Registration is public). The bulletin board lists every registered voter’s pk and handle. So “who registered” is public. The anonymity is only over which option each registered voter chose.

Issue 3 (No receipt-freeness). A voter who reveals x_i to a coercer can recompute $\sigma_i = H_0(I || R)^{x_i}$ and match it against the σ_j columns on each on-chain ballot to learn “this is my ballot”. This is the headline gap; see open problem 7.

Issue 4 (Network deanonymisation). Submitting a ballot reveals source IP and timing. The system has no built-in mitigation; voters would submit over Tor or a trusted relay in a real deployment.

Issue 5 (Voter device compromise). Malware with sk trivially impersonates the voter. Outside the cryptographic scope — needs hardware attestation.

13.2 The precise verifiability claim

What we provide. Given the bulletin board, the cohort public keys, and (if declared) the witness public keys: any auditor can run `evote audit` and:

1. Detect any tampering of the on-chain history.
2. Verify every ballot was OTRS-valid against the published ring and election ID.
3. Identify every voter who cast two distinct ballots and exclude their ballots from the tally.
4. Recompute the official tally and refute any dishonest cohort claim.
5. Detect any cohort equivocation, given honest witnesses.

The audit is *deterministic* and *public*: the same log gives the same result for any auditor anywhere.

Where this breaks down.

Issue 6 (End-to-end voter inclusion). A voter cannot cryptographically prove that *their specific* ballot was tallied without revealing their identity. They can verify “ N ballots tallied”, not “mine was one of them”. The literature solution is designated-verifier proofs, which conflict with receipt-freeness.

Issue 7 (Cohort attestation is the eligibility oracle). The cohort can attest any pk they want, including fake ones. The cryptography cannot detect this — it’s an identity-proofing policy problem. Mitigation: publicly observable registration, external identity authority.

Issue 8 (Cohort controls timestamps). A unanimous cohort can backdate ballots. Mitigations: witness checkpoints inside the voting window, or an external time anchor (Roughtime). Not in v0.3.

Issue 9 (Equivocation evidence needs multiple honest witnesses). A single witness signing a divergent head is not enough. In a real deployment M should be moderately large (e.g., 5+) and witnesses should be institutionally diverse.

Issue 10 (Censorship-evidence $\neq$ censorship-resistance). v0.3 makes censorship publicly evident but cannot force the cohort to publish. A unanimously malicious cohort stalls the election. Fix: open problem 3.

13.3 The honest pitch

The sentence to defend in front of a security researcher:

v0.3 is the cryptographic core of a verifiable anonymous voting system, with the threshold-cohort and witness-federation layers that the academic literature treats as necessary infrastructure; receipt-freeness, censorship-resistance, and end-to-end voter-inclusion remain open problems for v0.4 and beyond.

If you can defend that sentence with the ten Issue boxes above, you have understood the project.

A Glossary

anonymity set

The set of public keys among which a ring signature could plausibly have come from. Equals the ring.

audit

The deterministic verification of an election, performed by anyone with the bulletin board and the cohort/witness public keys. Implemented in `voting/auditor.py`.

ballot

A single voter's choice plus their OTRS signature, submitted as a record to the bulletin board.

bulletin board

The append-only JSON-Lines file where the election history lives. Each line is one entry.

checkpoint

A signed attestation by a witness of the form “at log index i , the head hash is h ”. Stored in a sidecar file.

cohort

The N organisers of an election, each holding their own Ed25519 key. Any subset of size $\geq t$ can co-sign log entries.

discrete logarithm

The hard problem “given g and g^x , find x ”. Our security rests on this being infeasible in Ristretto255.

Ed25519

The Ed25519 signature scheme. Used for cohort and witness signatures.

election ID

A 32-byte random hex string that uniquely identifies an election. Also serves as the OTRS *issue* value.

entry

One line of the bulletin board: index, prev_hash, timestamp, payload, publisher signatures.

equivocation

A cohort publishing two divergent versions of the log. Detected by the witness federation.

Fiat–Shamir

The trick of replacing an interactive protocol’s random challenges by hashes of the transcript so far. Used inside OTRS.

group

A set with an associative, identity-having, inverse-having operation. Ristretto255 for us.

hash function

Deterministic recipe producing a fixed-size fingerprint. SHA-256 for us.

hash chain

A sequence of entries each containing the hash of the previous. Makes the sequence tamper-evident.

hash-to-curve

Mapping arbitrary bytes deterministically into a group element. RFC 9380 instantiation for us.

issue

The OTRS election identifier. Equal to the election ID.

linked

Trace outcome when two ballots come from the same voter on the same message (a replay).

OTRS

One-time Traceable Ring Signature. The cryptographic primitive powering anonymous ballots with double-vote detection.

publisher cohort

Same as “cohort”.

record

The typed JSON payload of a bulletin-board entry. Six kinds: Setup, Registration, Ring, Ballot, Closed, Tally.

ring

The ordered list of voter public keys for the OTRS scheme. Published after registration closes.

ring signature

A signature over a ring of public keys, proving “one of these people signed” without revealing which.

Ristretto255

A prime-order group of order $q \approx 2^{252}$ built on top of Curve25519. Our algebraic playground.

secret key / public key

The two halves of a public-key cryptographic identity. Secret stays private; public goes on the bulletin board.

threshold

The minimum number of cohort or witness members that must co-sign for validity (t for cohort, k for witnesses).

trace

The OTRS algorithm that compares two ballots and outputs “independent”, “linked”, or “double-sign with culprit position”.

trace tag

The deterministic group element $\sigma_i = h^{x_i}$ embedded in every signature. Same voter on the same election always produces the same tag.

witness

An independent party who emits signed checkpoints attesting to the state of the bulletin board head.

References

- [1] A. Scafuro and B. Zhang. One-time Traceable Ring Signatures. ESORICS, 2021.
- [2] S. Scott, S. Wood, R. Wahby, A. Faz-Hernández, N. Sullivan. Hashing to Elliptic Curves. RFC 9380, 2023.
- [3] C. Komlo, I. Goldberg. FROST: Flexible Round-Optimized Schnorr Threshold Signatures. SAC, 2020.
- [4] B. Laurie, A. Langley, E. Käsper. Certificate Transparency. RFC 6962, 2013.
- [5] R. Berbain et al. Sigsum: Signed and verifiable software transparency. 2024.
- [6] J. Groth and M. Kohlweiss. One-Out-of-Many Proofs. EUROCRYPT, 2015.
- [7] A. Juels, D. Catalano, M. Jakobsson. Coercion-resistant electronic elections. WPES, 2005.
- [8] M. Clarkson, S. Chong, A. Myers. Civitas. IEEE S&P, 2008.
- [9] X. Lu, M. H. Au, Z. Zhang. Raptor: a Practical Lattice-Based (Linkable) Ring Signature. ACNS, 2019.
- [10] S. Park, M. Specter et al. Going from bad to worse: from Internet voting to blockchain voting. Journal of Cybersecurity, 2021.